상위 1%의 AI 활용법: 프롬프트를 넘어 하네스와 루프를 설계하라

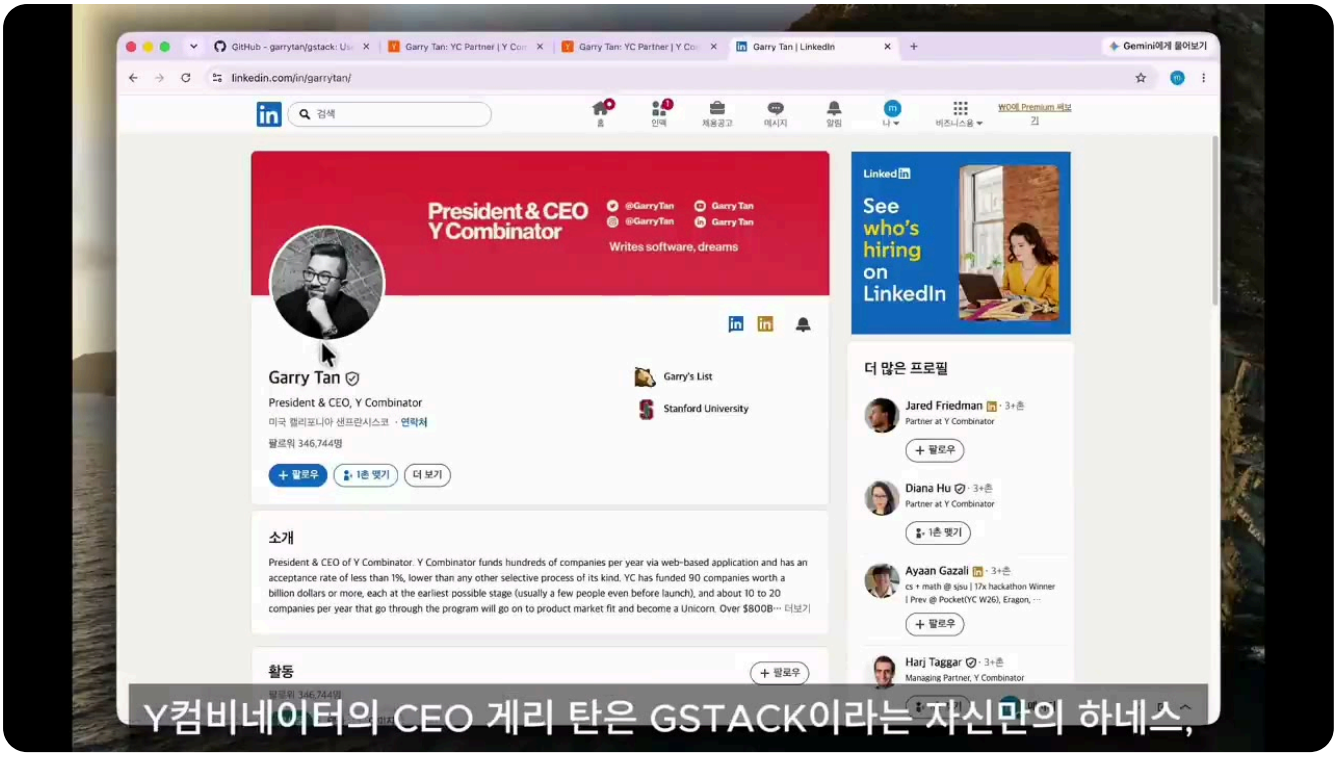
원본: 세계 최고의 전설적인 실리콘밸리 투자사 YC CEO의 상위 1% 하네스 · 마일드코드

Y 콤비네이터 CEO 게리 탄이 공개한 **G스택**을 계기로, 잘 묻는 **프롬프트**에서 AI가 일할 시스템을 설계하는 **하네스 엔지니어링**과 검증용 **루프 엔지니어링**으로 관심이 옮겨가고 있음을 설명한다. 핵심은 남의 하네스를 그대로 쓰지 말고 내 문제에 맞게 직접 만들고, 필요 없어진 장치는 걷어내며 유지보수하는 것이다. 말로 하는 부탁 대신 시스템으로 강제하는 장치를 하나씩 붙여 가라는 것이 영상의 결론이다.

- 상위 1% 사용자는 프롬프트를 다듬기보다 AI 에이전트의 역할과 작업 순서라는 시스템을 먼저 설계한다.
- 반드시 지켜야 할 규칙은 프롬프트의 부탁으로 두지 말고 훅, 커밋 검사, 배포 검사처럼 어길 수 없는 결정론적 장치로 강제한다.
- 하네스 엔지니어링은 AI의 실수를 발견할 때마다 환경 자체를 고쳐 같은 실수를 막는 것이며, 루프 엔지니어링은 코드를 쓰는 루프 옆에 검사 루프를 두는 것이다.
- 남이 만든 하네스에는 그 사람의 가정이 들어 있으므로 그대로 쓰지 말고 만든 이유를 배워 직접 만들고, 모델이 좋아지면 불필요한 장치를 걷어낸다.
- 'AI가 다 됐다'는 말 대신 검사 통과를 완료의 정의로 삼고, 위험 작업 차단과 반복 잔소리의 역할화를 먼저 적용한다.

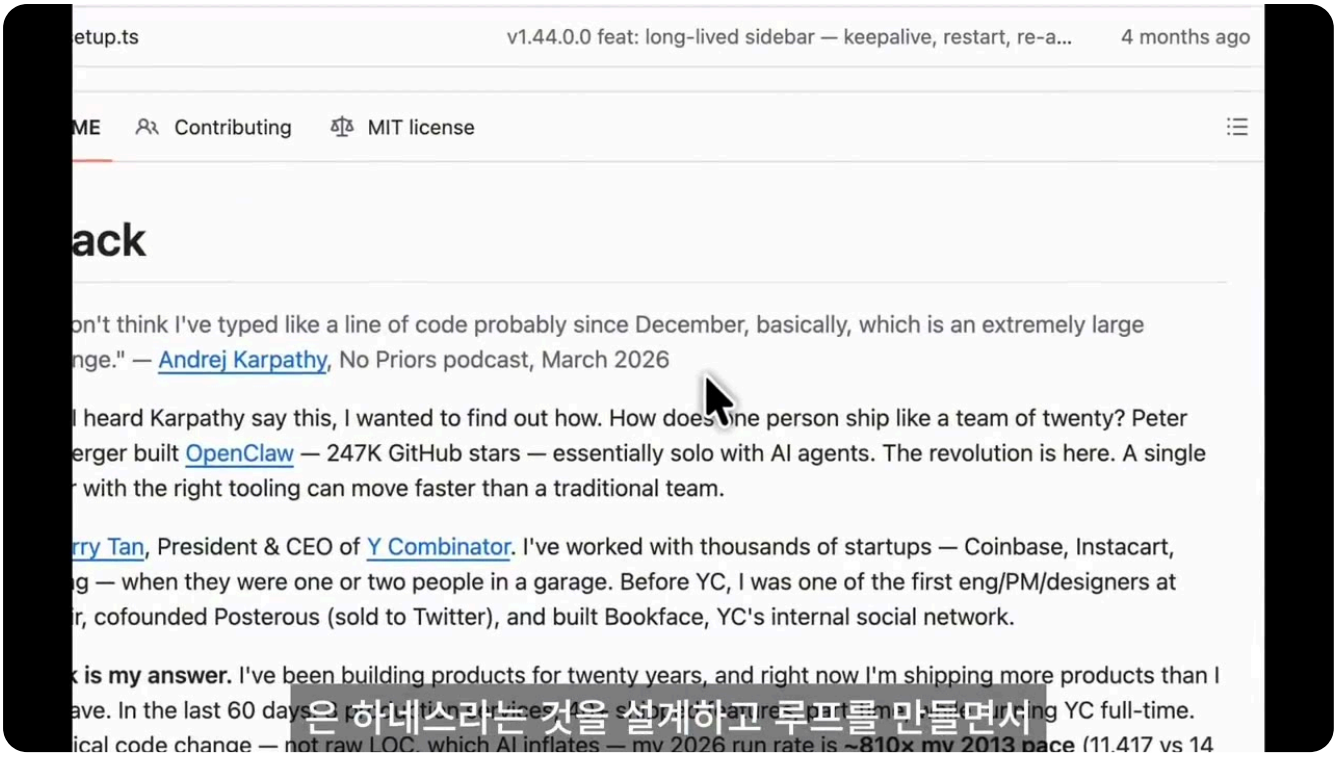
1. G스택과 역할 기반 AI 팀 설계

1.1. 게리 탄과 G스택 소개



1. Y 콤비네이터 CEO 게리 탄이 자신만의 하네스이자 에이전트 팀인 G스택을 오픈 소스로 공개했다. [00:57]
2. 게리 탄은 샘 알트먼에 이어 Y 콤비네이터를 이끌고 있으며, 스탠포드 컴퓨터공학 졸업 후 팔란티어에서 일하고 포스테리우스를 창업해 트위터에 엑싯한 인물이다. [01:11]
3. 본인이 직접 쓰는 워크플로우를 깃허브에 공개하자 10만 명이 넘는 사람이 스타를 눌러 세계적으로 화제가 되었다. [01:29]

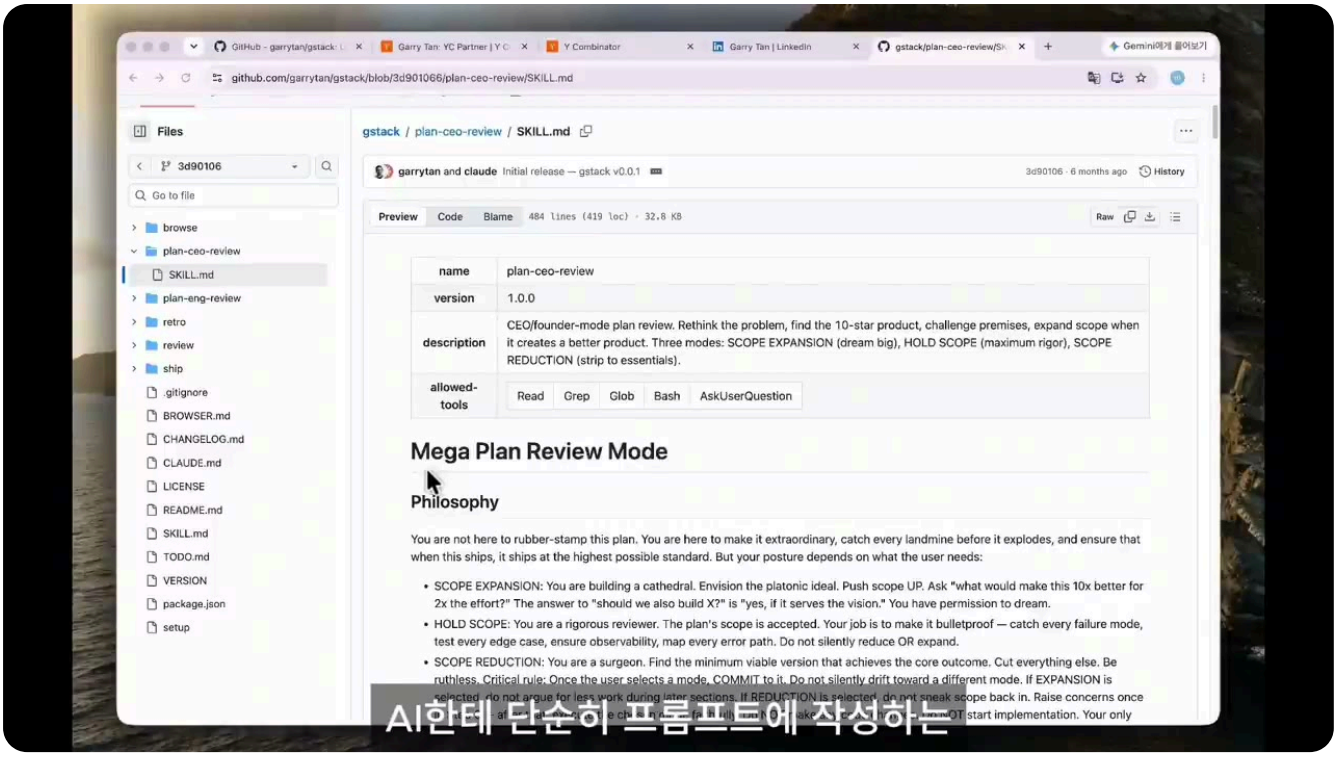
1.2. 프롬프트가 아닌 시스템을 먼저 설계



1. 영상이 꼽는 가장 중요한 교훈은 프롬프트를 잘 쓰려고 애쓰지 않는 것이며, 상위 천재들은 AI가 일할 역할과 순서의 시스템을 먼저 설계한다. [00:14] [01:47]
2. 이들은 AI가 의도와 목적에 맞게 스스로 판단해 최고의 결과물을 내도록 하네스를 설계하고 루프를 만든다. [00:27]
3. G스택에는 제품 총괄 CEO, 아키텍처를 감독하는 엔지니어링 매니저, 홍보와 판매를 맡는 마케터 등 23개 역할의 에이전트가 미리 구성되어 있다. [02:06]
4. 각 역할은 계획, 제작, 상호 리뷰, 테스트, 배포, 회고의 사이클을 도는 루프로 연결되어 있다. [02:06] [02:24]
5. 프롬프트 고민에 시간을 쓰기보다 팀에서 누가 어떤 책임과 역할을 맡을지부터 정의하라고 권한다. [02:24] [02:41]

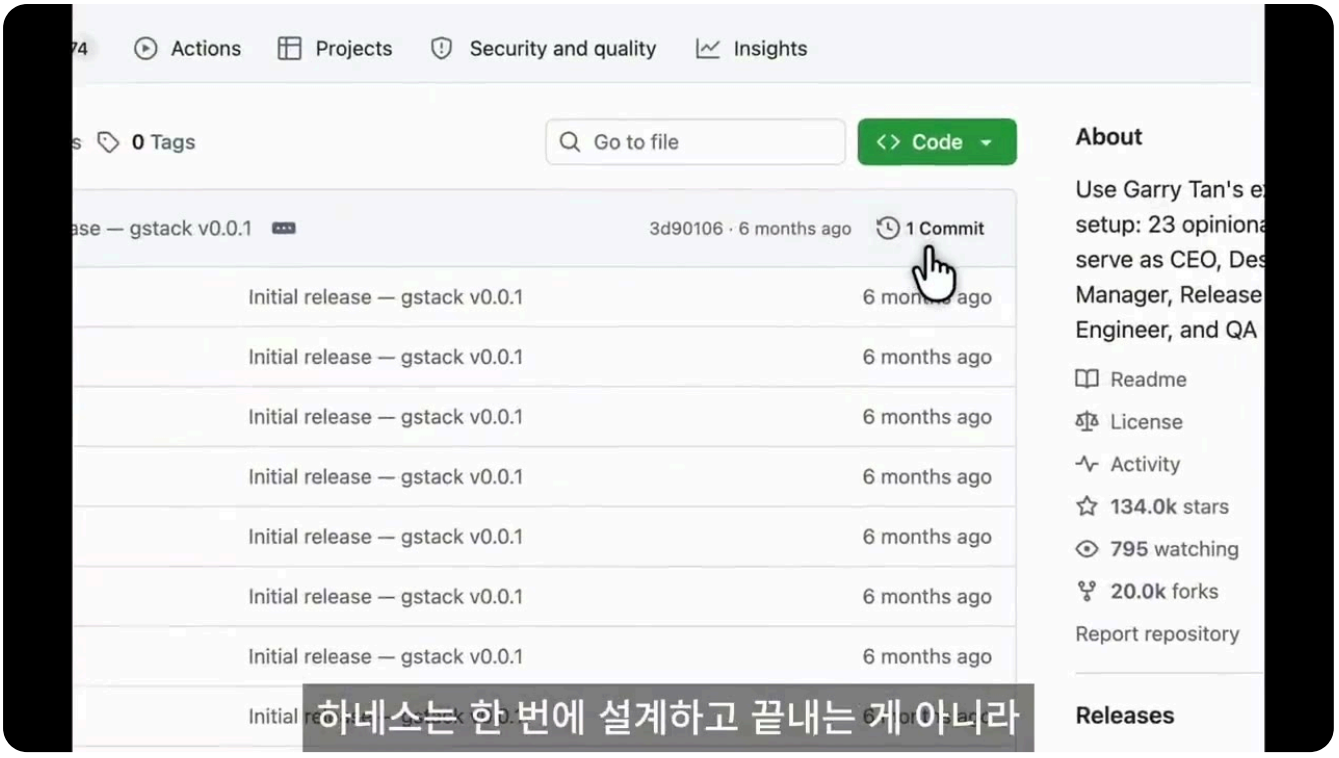
2. 부탁이 아닌 강제: 결정론적 장치

2.1. 부탁과 강제의 차이



1. 역할만 나눈다고 의도대로 지켜지지 않으므로, 반드시 지켜야 하는 규칙은 프롬프트의 부탁이 아니라 어길 수 없게 시스템으로 강제해야 한다. [02:46] [03:06]
2. 모든 것을 기계적으로 만들 수는 없고 그래서도 안 되는데, 그러면 똑똑한 AI 모델 성능의 절반도 쓰지 못하기 때문이다. [03:06] [03:25]
3. G스택의 CEO 스킬은 '도장이나 찍어 주려고 존재하는 게 아니다'라고 시작하며 AI의 사고와 추론을 요청하는데, 페르소나를 주고 판단하게 하는 것도 좋은 방법이다. [03:25] [03:43]
4. 반면 위험 명령 차단 스킬은 터미널 명령 실행 전에 스크립트가 먼저 돌아 폴더를 통째로 지우는 명령을 그 자리에서 막는 훅으로, 부탁이 아닌 강제 시스템이다. [03:43] [04:01]

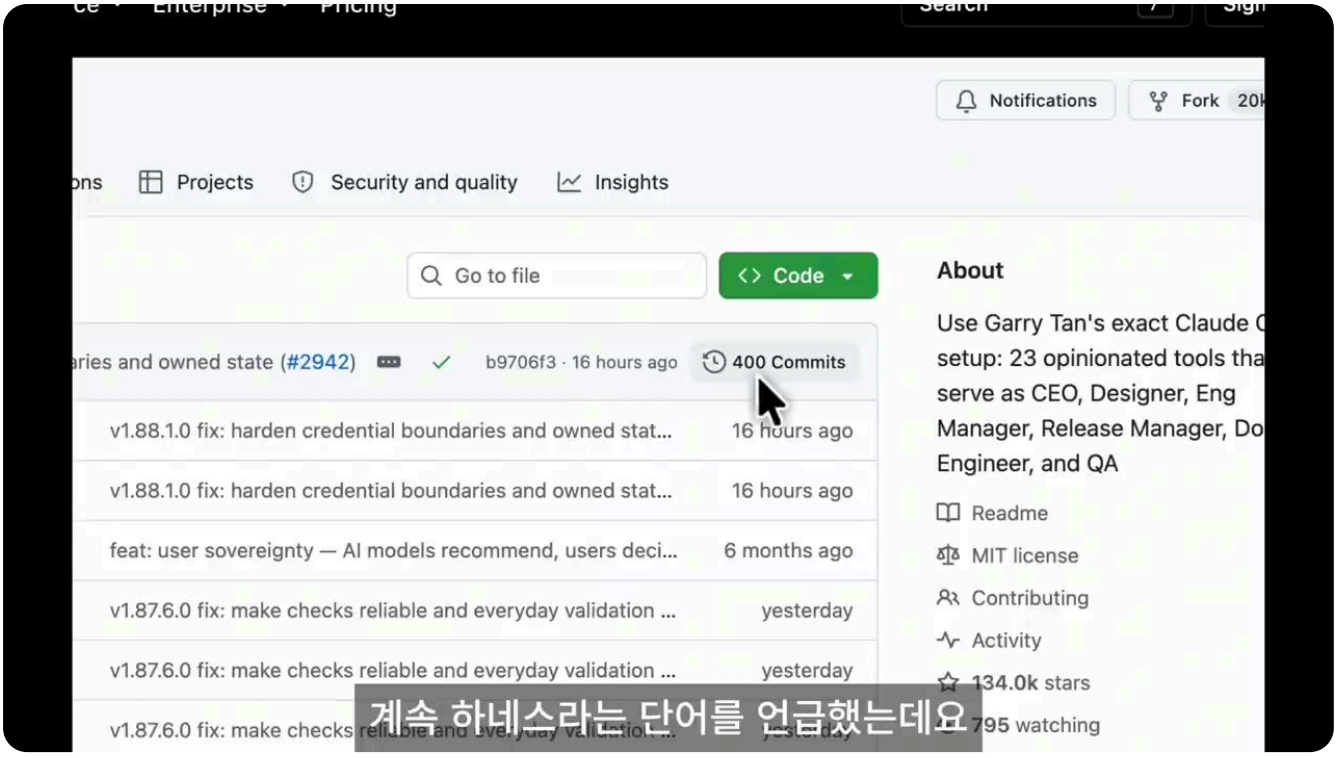
2.2. 테스트와 검증 계층, 그리고 점진적 구축



1. 게리 탄이 말도 안 되는 속도의 성과 비결로 꼽은 것도 에이전트 역할이나 스킬이 아니라 테스트와 검증을 하는 시스템 계층이었다. [04:01]
2. 매번 말로 부탁했지만 잘 안 지켜지는 명령 하나를 골라 절대 어길 수 없는 방어 장치로 바꾸는 것부터 시작하라고 조언한다. [04:01] [04:19] [04:26]
3. 게리 탄도 처음부터 모두 갖추지 않았고, 작업하면서 취향에 맞게 하나씩 붙여 지금처럼 발전했다. [04:26] [04:34]
4. 하네스는 한 번에 설계하고 끝내는 것이 아니라 AI의 실수와 반복 작업을 최적화하며 하나씩 붙여 가는 것이다. [04:34]

3. 하네스 엔지니어링과 루프 엔지니어링

3.1. 하네스 엔지니어링의 본질



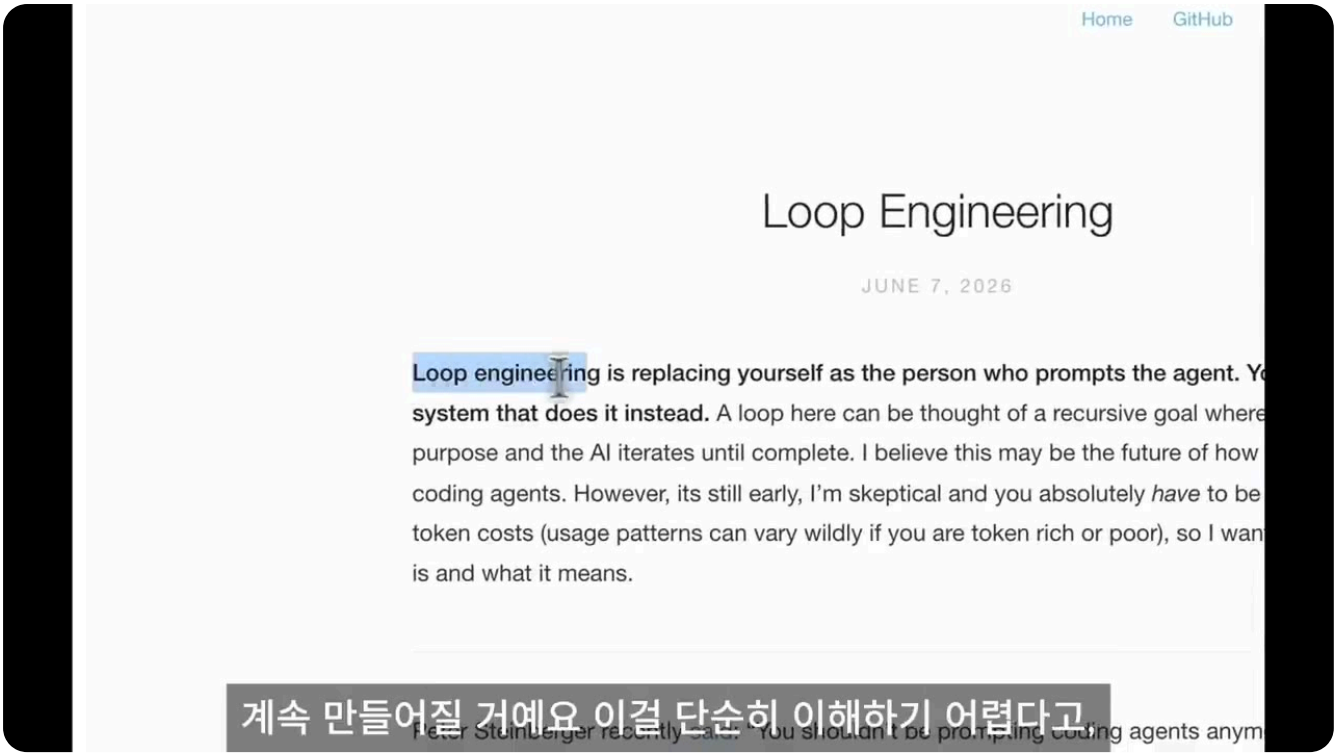
1. AI 에이전트가 우리가 만든 시스템 안에서 작업하게 만드는 것을 하네스 엔지니어링이라 하며, 올해 초 미첼 하시모토가 이 이름으로 정리했다. [04:47]
2. 의도는 단순해서, 에이전트의 실수를 발견하면 다시는 못 하도록 환경 자체를 고쳐 두는 것이 전부다. [04:47] [05:06]
3. 화자의 규칙 파일과 자동 검사 도구도 모두 AI가 특정 실수나 반복 작업에 실패한 것을 해결하려고 하나씩 추가된 것들이다. [05:06]

3.2. 루프 엔지니어링과 검증 루프



1. 루프 엔지니어링의 핵심은 서비스를 만드는 메인 루프 옆에 검사를 하는 루프를 두는 것이며, 올해 6월 에디 오스만이 이름을 붙였다. [06:30]
2. 클로드 코드를 만든 엔트로픽도 코드를 쓰는 루프에는 코드를 검사하는 루프가 필요하다고 말한다. [06:30] [06:48]
3. 사람이 코드를 직접 검수하는 것이 사실상 불가능해지면서, 사람이 검수하던 자리를 AI로 대체하자는 발상이 나왔다. [06:48]
4. 화자는 작년까지 클로드 코드가 10분 만에 쓴 코드를 세 시간씩 눈으로 검수했지만 그 방법은 지속 가능하지 않았다. [06:59]
5. G스택도 지난달 통과 조건이 충족될 때까지 AI가 스스로 문제를 찾아 고치는 검증 루프를 붙였고, 검증 루프 없이는 루프를 돌리지 말라는 메시지도 남겼다. [06:59] [07:17]

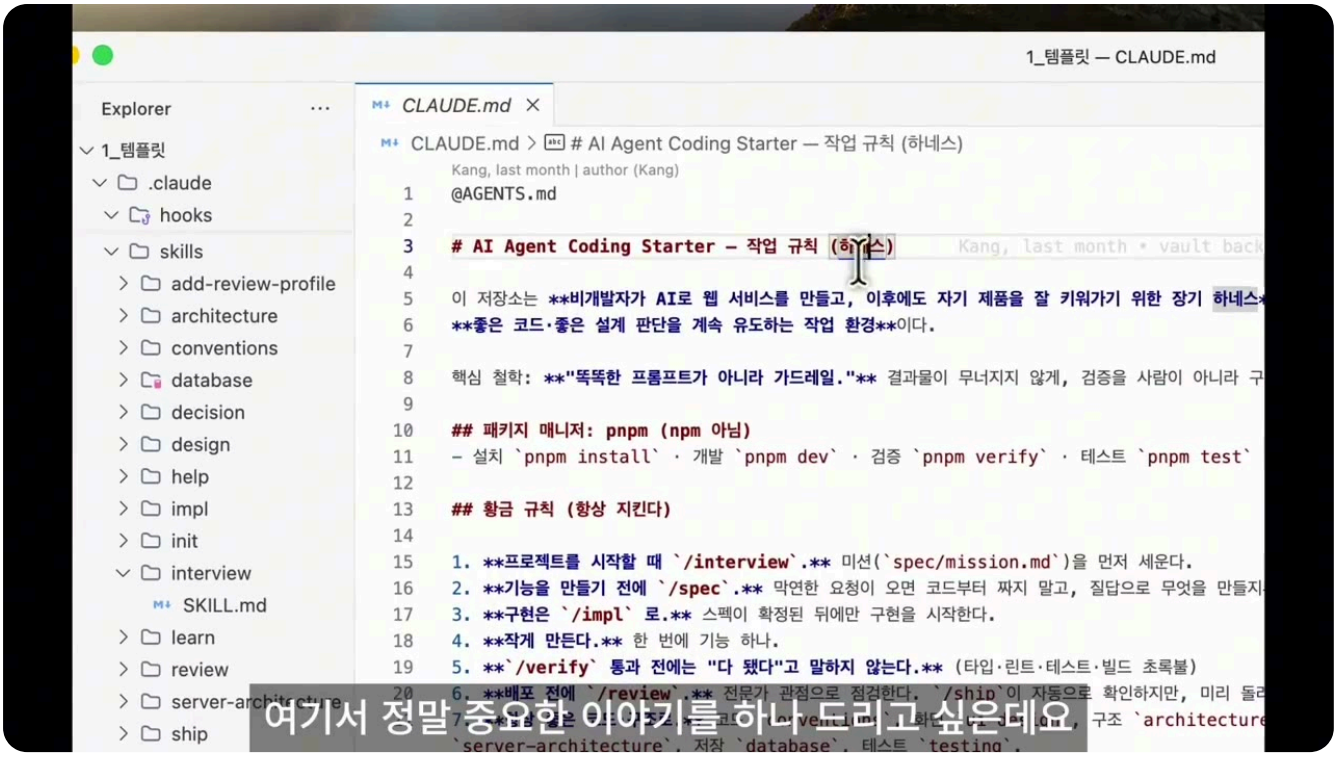
3.3. 유행어를 대하는 태도



1. 프롬프트, 하네스, 루프 엔지니어링 같은 새 용어가 나올 때마다 유행이라고 배척하지도, 앞선 것은 끝났다고 극단적으로 말하지도 말고 본질을 보라고 한다. [05:24] [05:44] [05:54]
2. 프론티어의 관심사가 이동하는 방향과 거기서 얻을 배움이 중요하며, 이해하기 어렵다고 외면하지 말고 건질 본질을 찾는 마인드로 접근하라고 한다. [05:54] [06:10]

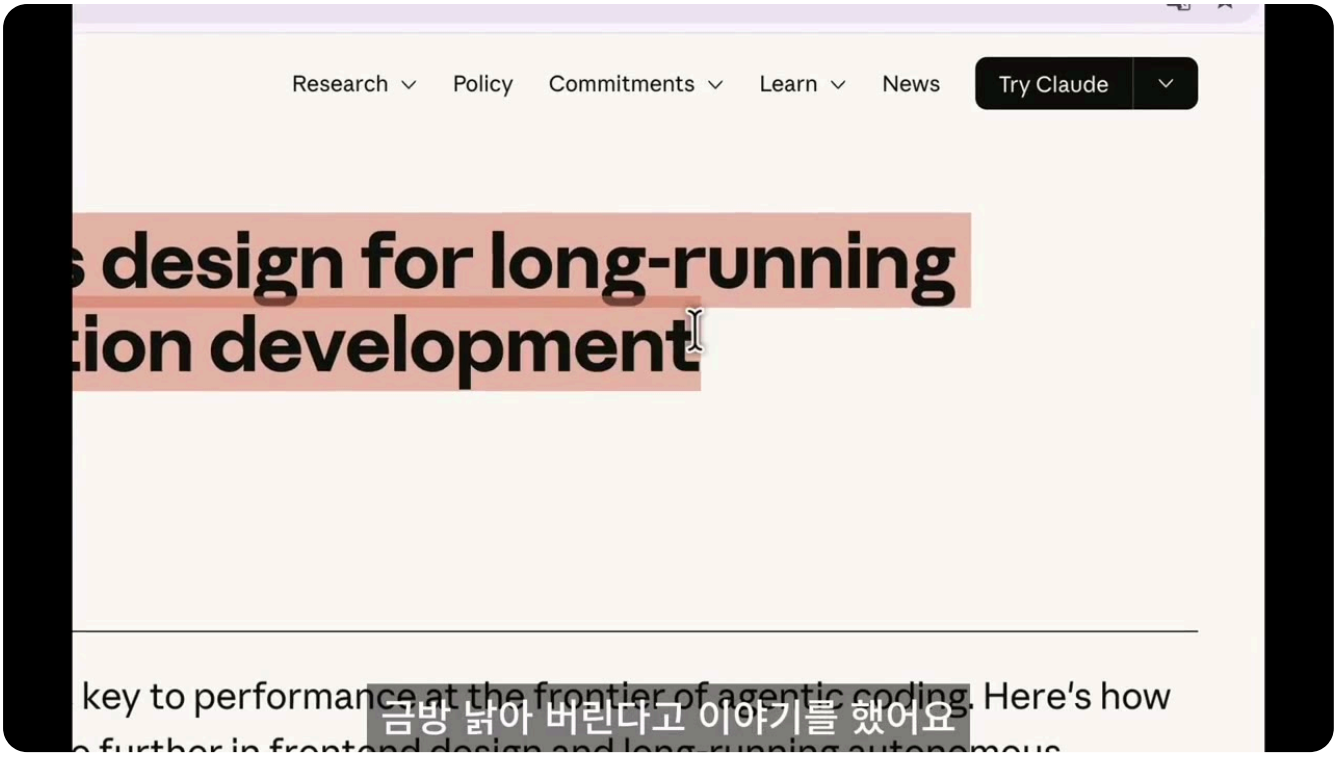
4. 남의 하네스를 그대로 쓰지 말고 직접 만들어라

4.1. 내 문제에 맞는 하네스



1. 게리 탄의 장치 목록은 그의 경험과 취향으로 만든 것이라 다른 사람의 하네스는 내 문제를 제대로 해결해 주지 않는다. [05:06] [05:24]
2. G스택을 그대로 써 본 사람이 이슈 게시판에 남의 세팅은 영감만 받고 직접 다시 만들어 쓰는 게 맞다는 글을 올렸다. [07:17] [07:36]
3. 화자도 클로드 코드 초기에 남들이 추천한 플러그인과 스킬을 여러 개 깔았지만 결국 맞지 않는 것이 많았다. [07:36] [07:50]
4. 커뮤니티 스킬은 그대로 가져다 쓰지 말고 왜 만들었는지, 어떤 문제를 어떻게 풀었는지를 배워 가져오라고 한다. [08:10] [08:29]

4.2. 하네스의 비용과 유지보수



1. 엔트로픽에 따르면 하네스의 모든 구성 요소에는 모델이 혼자 못 하는 것에 대한 가정이 들어 있고, 그 가정은 모델이 좋아지면 금방 낡는다. [07:50] [08:10]
2. 하네스가 많이 붙을수록 토큰 비용이 증가하므로 필수적인 시스템만 남기는 것도 중요한 설계 원칙이다. [08:29] [08:46]
3. 화자는 처음 프로젝트 규칙을 20개 넘게 만들었다가 불필요한 규칙이 좀비처럼 남는 경험을 했다. [08:52]
4. 모델과 클로드 코드 기능이 발전해 필요 없어진 하네스도 많아, 엔트로픽이 새 모델 출시 때 자체 하네스를 줄였듯 주기적으로 걷어내야 한다. [09:03] [12:48]

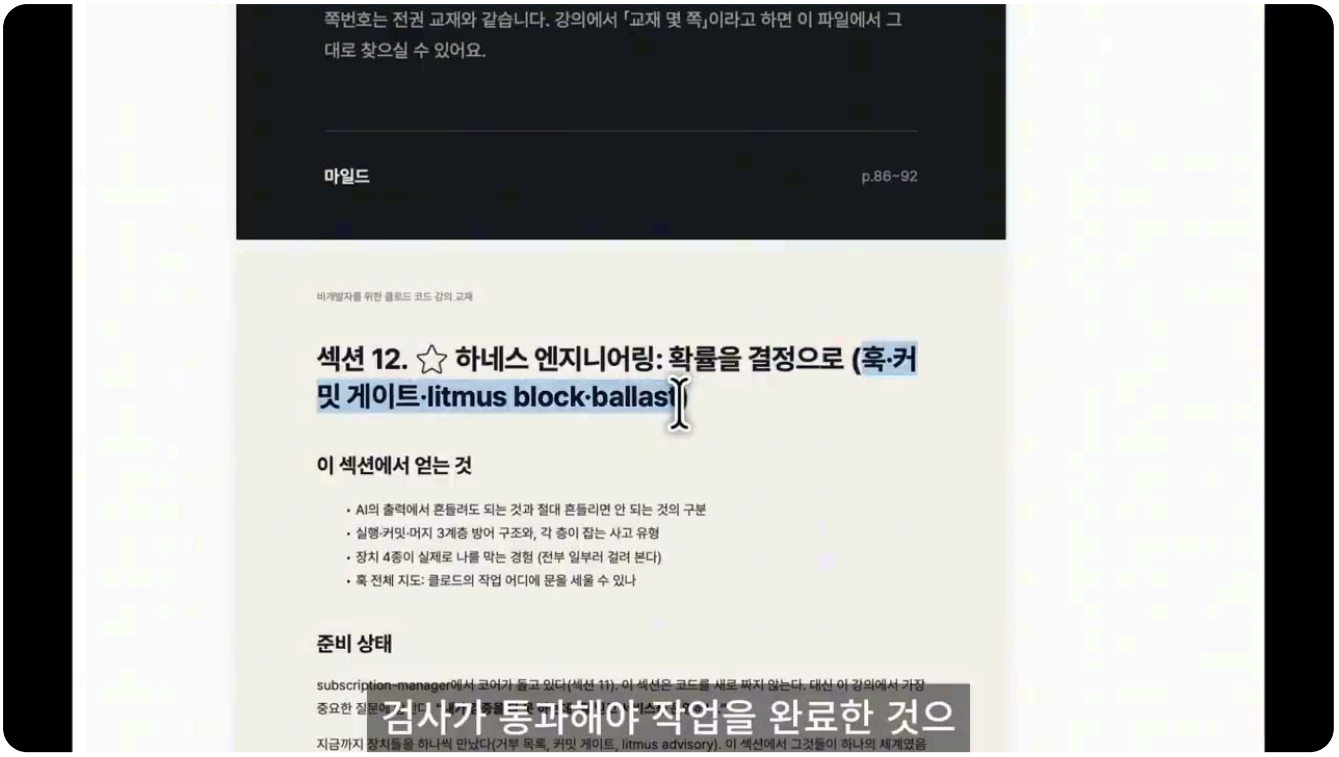
5. 실전 적용: 검증이 필요한 세 지점

5.1. 화자의 실제 장치들

1. 첫 장치는 클로드 코드가 일을 끝내려는 순간 검사를 돌리고, 통과하지 못하면 그 턴 자체가 끝나지 않게 한 것이다. [09:40] [10:18]
2. 화자는 시리즈 B AI 스타트업의 풀스택 프로덕트 엔지니어이자 엣지링크라는 서비스를 혼자 운영하며, 기능 하나가 깨지면 곧 장애와 매출 손실이라 이런 장치가 필요했다. [09:59] [10:18]

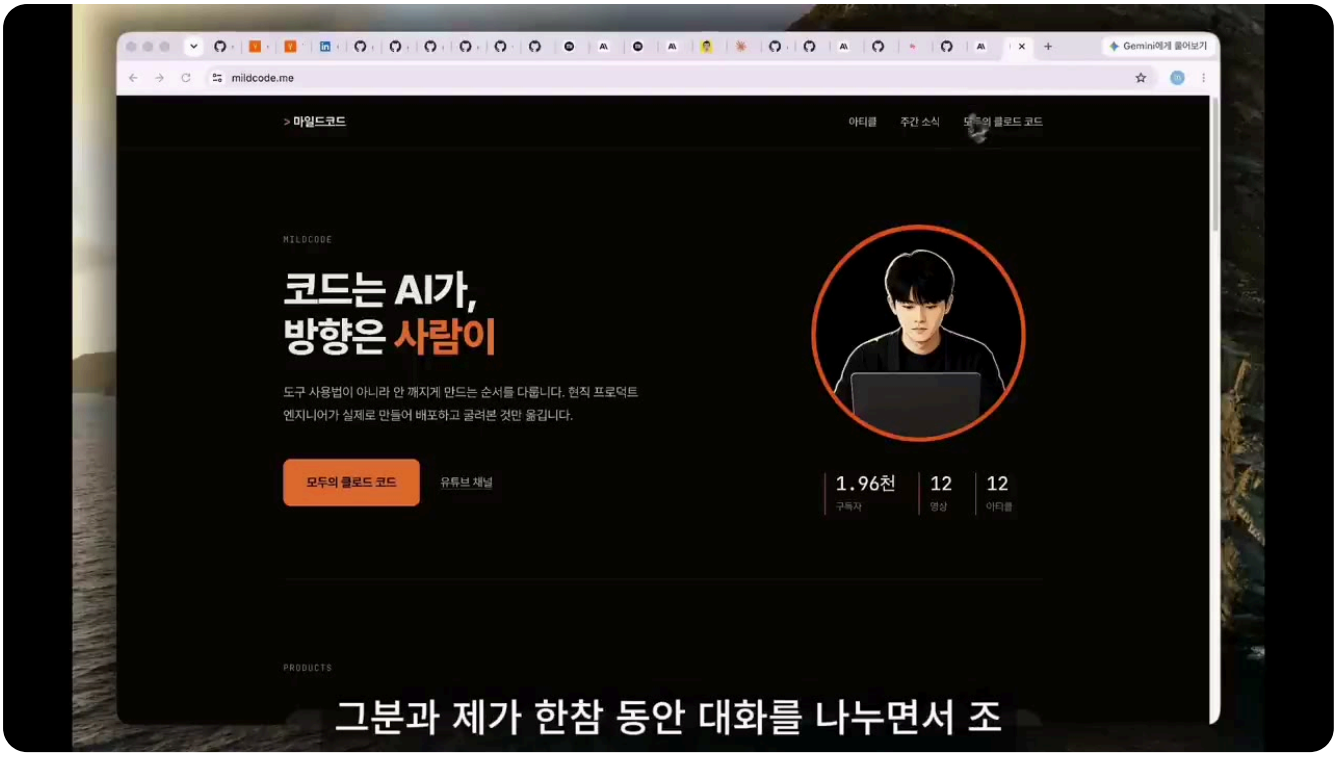
3. 유저 테이블에 컬럼을 추가했는데 기존 데이터에 그 컬럼이 없어 앱이 멈추는 사고를 막으려고, 커밋 시점에 데이터베이스 변경 이력이 없으면 저장 자체가 안 되도록 검사를 걸었다. [10:36] [10:55] [11:10]
4. 이 검사 하나로 100번이 훨씬 넘는 데이터베이스 구조 변경을 사고 없이 운영했고, 배포도 자동 검사가 통과해야만 가능하게 했다. [11:10] [11:27]

5.2. 검증해야 할 세 지점



1. AI가 다 됐다는데 열어 보면 깨져 있을 때 고칠 것은 프롬프트가 아니라 완료의 정의이며, 무엇이 통과하면 완료인지 정해 검사 통과로만 판정하게 한다. [09:40] [11:51]
2. 지우면 안 되는 것을 지우거나 손대면 안 되는 데이터를 건드리는 일은 부탁으로 두면 언젠가 뚫리므로 실행 자체를 강제로 막는다. [11:51] [12:11]
3. 게리 탄의 훅과 화자의 커밋 시점 검사가 바로 이 원칙의 사례다. [12:11]
4. 매번 반복하는 잔소리는 말로 하지 말고 역할로 넘기며, G스택의 리뷰 스킬이 그 구조다. [12:29]

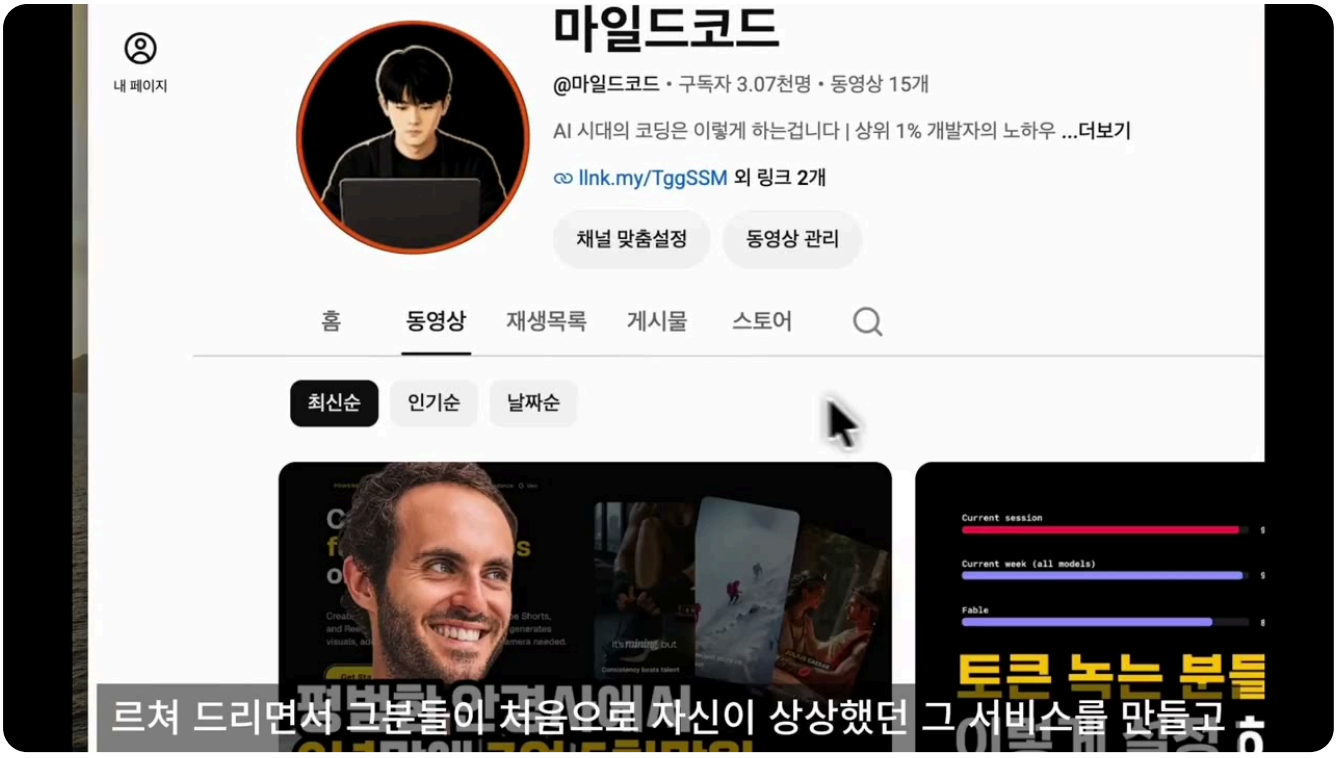
5.3. 기능을 추가할 때마다 깨지는 문제



1. 한 구독자는 기능을 추가할 때마다 다른 곳이 깨지고 버그 하나를 고치면 다른 곳이 터져 바이브 코딩이 재미없어졌다고 호소했다. [12:48] [13:07]
2. 유명 스킬과 플러그인을 그대로 설치해도 규모가 커지면 멀쩡하던 것이 깨지는 문제는 스스로 워크플로우를 정의하고 하네스를 설계해야 풀린다. [14:27] [14:42]
3. G스택의 슬래시 리뷰처럼, 내 서비스에서 자주 생기는 문제 지점을 대신 봐주는 검토자를 직접 만들어야 한다. [14:42] [15:00]

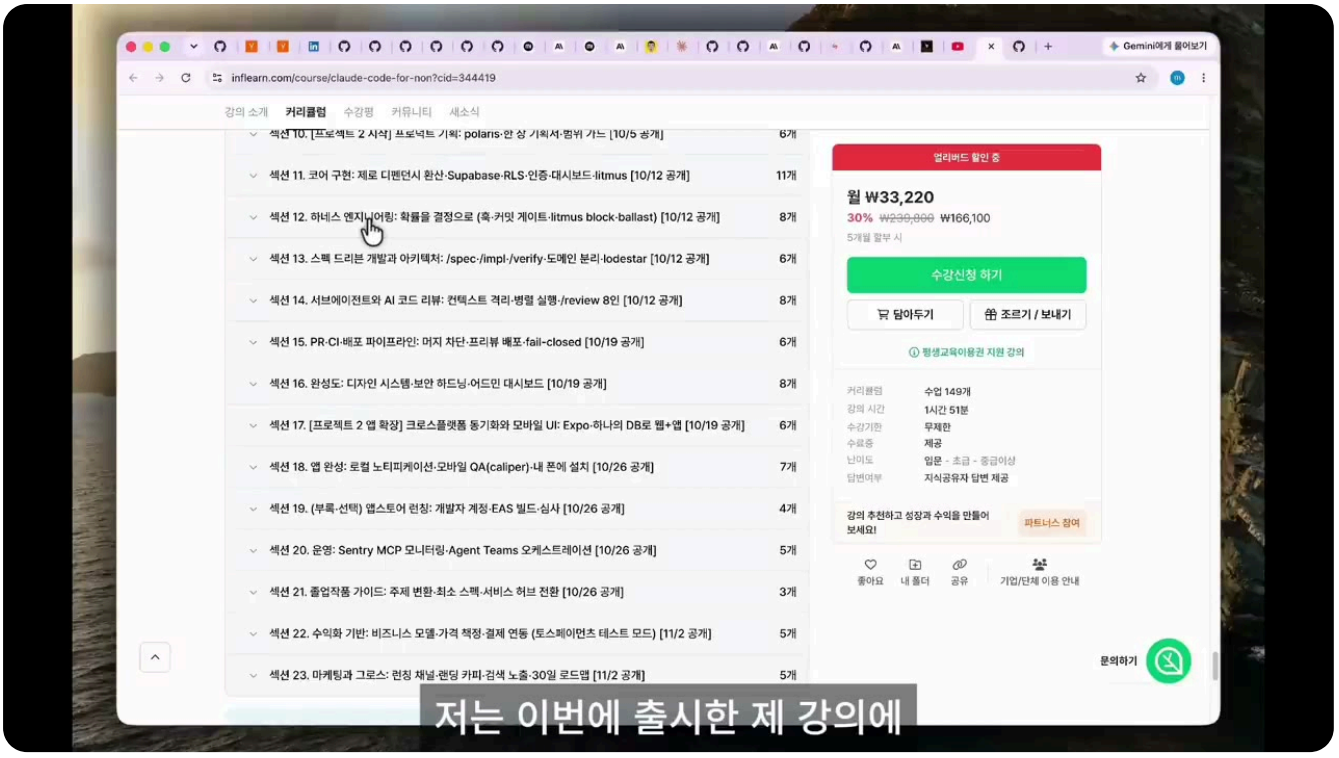
6. 유튜브를 하는 이유와 강의 안내

6.1. 채널을 시작한 동기



1. 가까운 사람들에게 클로드 코드로 서비스를 만들고 수익화하는 법을 가르치며, 상상하던 서비스를 처음 만든 그들이 기뻐하는 모습을 본 것이 계기였다. [13:21]
2. 더 많은 사람이 내 문제를 소프트웨어로 직접 해결하는 쾌감을 느끼기를 바란다. [13:21] [13:41]
3. 코딩은 어렵다고 생각하던 사람들이 이제 앱과 서비스, 업무 시스템 프로그램을 만들고 있다. [13:49]

6.2. '모두의 클로드 코드' 강의



1. 강의는 클로드 코드 설치부터 시작해 한 번도 써 보지 않은 사람도 따라올 수 있고, 입문에서 시작해 고급자 수준까지 올라가는 약 22시간 분량이다. [15:00] [15:49]
2. 하네스 장치를 직접 걸어 보고 플러그인을 만들며 웹·앱 결제 운영과 수익화까지 한 과정으로 잇는 강의는 이것뿐이라고 소개한다. [15:29] [15:49]
3. AI 기술 변화에 맞춰 앞으로도 계속 업데이트할 예정이며, 강의 링크는 고정 댓글에 있다. [15:29] [16:01]